

Documentacion del Sistema

Sistema de Control de Asistencias y Agenda Escolar desarrollado en Flask con Python.

Resumen general

- Proyecto basado en Flask y Flask-SQLAlchemy.
- Base de datos local: SQLite (sistema_academico.db).
- Usuarios con roles: directivo y personal.
- Control de asistencia mediante código QR y registro en tiempo real.
- Agenda institucional con eventos y calendario.

Modelos de datos

Usuario:

- id: entero, primary key.
- nombre, apellido, correo unico.
- password_hash para acceso seguro.
- telefono, fecha_nacimiento opcionales.
- rol: directivo o personal.

Evento:

- id: entero, primary key.
- titulo, fecha_inicio y fecha_fin.
- tipo: clase, feriado, institucional.
- profesor_id: FK opcional a Usuario.

AsistenciaDiaria:

- id: entero, primary key.
- usuario_id: FK a Usuario.
- fecha (dia del registro).
- hora_registro (datetime).
- estado: A tiempo, Tarde, Ausente, Inasistencia Reportada.
- motivo, observaciones, archivo_adjunto.

Configuracion:

- id: entero, primary key.
- hora_entrada_oficial en formato HH:MM.

Rutas principales

- / : login y redireccion por rol.
- /directivo: panel del directivo.
- /personal: panel del personal.
- /directivo/usuarios: gestion de usuarios.
- /directivo/planificacion: gestion de actividades.
- /directivo/configuracion_vista: ajuste de hora oficial.
- /directivo/configurar_hora: actualiza la hora de entrada.
- /directivo/crear_usuario: crea usuario nuevo.
- /directivo/editar_usuario/<id>: edita usuario existente.
- /directivo/eliminar_usuario/<id>: elimina usuario.
- /directivo/generar_qr: genera QR de registro de asistencia.
- /asistencia/registrar: registra asistencia tras escanear el QR.
- /api/eventos: devuelve eventos en JSON para el calendario.
- /api/evento/<id>: obtiene un evento por su identificador.
- /directivo/crear_evento: guarda un nuevo evento.
- /directivo/modificar_evento: modifica un evento.
- /logout: cierra sesion.
- /personal/registrar_inasistencia: reporta una inasistencia justificada.

Flujo de QR y asistencia

- 1. El directivo genera el QR en /directivo/generar_qr.
- 2. El QR codifica el enlace de registro de asistencia.
- 3. El telefono escanea el QR y abre la URL.
- 4. Si no hay sesion activa, el sistema redirige al login con "next".
- 5. Tras login, el usuario vuelve a /asistencia/registrar y se guarda la asistencia.
- 6. El estado se calcula comparando la hora actual con la hora oficial.
- 7. Si ya se registro hoy, muestra mensaje y redirige al panel.

Notas de implementacion: se usa request.host_url para generar la URL accesible por el movil, y se pasa el parametro next al login para mantener la navegacion original.

Plantillas HTML importantes

- login.html: formulario de acceso y envio de "next".
- director.html: dashboard del directivo y enlace al QR.
- personal.html: dashboard del personal y modal del escaner QR.
- qr_panel.html: vista de QR para imprimir o mostrar.
- director_usuarios.html: lista y gestion de usuarios.
- director_configuracion.html: ajuste de hora de entrada oficial.
- director_planificacion.html: calendario y eventos.
- reporte.html: reporte filtrable de asistencias.

Recomendaciones de despliegue

- Ejecutar el servidor con python app.py en el directorio del proyecto.
- Asegurarse de que el movil y el servidor esten en la misma red local.
- Crear la carpeta static/uploads si se quiere subir archivos de inasistencia.
- Mantener la clave secreta segura en produccion.
- Verificar que el puerto 8080 este accesible para el dispositivo movil.